293 lines (233 loc) · 10.7 KB

Code Blame Raw

```python
import os
import random
import sys
import math
import pygame

# ワーキングディレクトリを作業ファイルの場所に設定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# 定数定義
SCREEN_WIDTH = 900
SCREEN_HEIGHT = 600
FPS = 60
TIME_LIMIT = 120  # 制限時間（秒）

# 色の定義（RGB）
COLOR_BG = (50, 160, 50)        # 緑色（フィールド）
COLOR_PLAYER = (0, 100, 255)    # 青色（プレイヤー）
COLOR_ENEMY = (255, 50, 50)     # 赤色（敵）
COLOR_WHITE = (255, 255, 255)  # 白色
COLOR_RED = (200, 0, 0)         # 強調用赤


class Player:
    """プレイヤーキャラクターの移動、HP、描画を管理するクラス

    def __init__(self, x: float, y: float) -> None:
        """プレイヤーの初期位置やステータスを設定する"""
        self.x: float = x
        self.y: float = y
        self.radius: int = 15
        self.speed: float = 4.0
        self.hp: int = 100
        self.max_hp: int = 100
        self.invincible_timer: int = 0  # 被弾後の無敵時間

    def update(self, keys: pygame.key.ScancodeWrapper) -
        """キー入力を受けて移動処理を行い、画面外への逸脱を防
        dx: float = 0.0
        dy: float = 0.0

        # WASDキーによる入力判定
```

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols R

- const SCREEN_WIDTH
- const SCREEN_HEIGHT
- const FPS
- const TIME_LIMIT
- const COLOR_BG
- const COLOR_PLAYER
- const COLOR_ENEMY
- const COLOR_WHITE
- const COLOR_RED
- class Player
  - func __init__
  - func update
  - func draw
  - func get_pos
  - func get_hp
  - func take_damage
- class Enemy
  - func __init__
  - func update
  - func draw
  - func get_pos
- class Game
  - func __init__

```
            if keys[pygame.K_w]:
                dy -= 1.0
            if keys[pygame.K_s]:
                dy += 1.0
            if keys[pygame.K_a]:
                dx -= 1.0
            if keys[pygame.K_d]:
                dx += 1.0

            # 斜め移動時の速度が速くならないように正規化（1 / √2
            if dx != 0.0 and dy != 0.0:
                dx *= 0.7071
                dy *= 0.7071

            # 位置の更新
            self.x += dx * self.speed
            self.y += dy * self.speed

            # 画面外に出ないように制限
            if self.x - self.radius < 0:
                self.x = self.radius
            elif self.x + self.radius > SCREEN_WIDTH:
                self.x = SCREEN_WIDTH - self.radius

            if self.y - self.radius < 0:
                self.y = self.radius
            elif self.y + self.radius > SCREEN_HEIGHT:
                self.y = SCREEN_HEIGHT - self.radius

            # 無敵タイマーのカウントダウン
            if self.invincible_timer > 0:
                self.invincible_timer -= 1

        def draw(self, screen: pygame.Surface) -> None:
            """プレイヤーを描画する（無敵時間中は点滅表現を行う）
            # 無敵時間中は点滅させる（5フレームごとに表示・非表示
            if self.invincible_timer > 0 and (self.invincibl
                return

            pygame.draw.circle(screen, COLOR_PLAYER, (int(se
            pygame.draw.circle(screen, COLOR_WHITE, (int(sel

        def get_pos(self) -> tuple[float, float]:
            """プレイヤーの現在座標を返す"""
            return (self.x, self.y)

        def get_hp(self) -> int:
            """プレイヤーの現在のHPを返す"""
            return self.hp

        def take_damage(self, amount: int) -> None:
            """ダメージを受け、無敵時間を付与する"""
            if self.invincible_timer == 0:
                self.hp -= amount
                if self.hp < 0:
                    self.hp = 0
                self.invincible_timer = 60  # 1秒間（60フレー

```

func reset_game

func run

func spawn_enemy

func check_collisions

func draw

```
101
102 class Enemy:
103     """敵キャラクターの移動と描画を管理するクラス"""
104
105     def __init__(self, x: float, y: float) -> None:
106         """敵の初期位置を設定する"""
107         self.x: float = x
108         self.y: float = y
109         self.radius: int = 12
110         self.speed: float = 2.2
111
112     def update(self, player_x: float, player_y: float) -
113         """プレイヤーの方向に向かって移動する"""
114         dx: float = player_x - self.x
115         dy: float = player_y - self.y
116         distance: float = math.hypot(dx, dy)
117
118         # プレイヤーに向かって単位ベクトルを掛けて移動
119         if distance > 0:
120             dx /= distance
121             dy /= distance
122             self.x += dx * self.speed
123             self.y += dy * self.speed
124
125     def draw(self, screen: pygame.Surface) -> None:
126         """敵を描画する"""
127         pygame.draw.circle(screen, COLOR_ENEMY, (int(sel
128         pygame.draw.circle(screen, COLOR_WHITE, (int(sel
129
130     def get_pos(self) -> tuple[float, float]:
131         """敵の現在座標を返す"""
132         return (self.x, self.y)
133
134
135 class Game:
136     """ゲーム全体の状態、時間、スコア、当たり判定を管理するク
137
138     def __init__(self) -> None:
139         """Pygameとゲーム変数を初期化する"""
140         pygame.init()
141         self.screen: pygame.Surface = pygame.display.set
142         pygame.display.set_caption("SURVIVAL ARENA")
143         self.clock: pygame.time.Clock = pygame.time.Cloc
144         self.font: pygame.font.Font = pygame.font.SysFon
145         self.large_font: pygame.font.Font = pygame.font.
146
147         self.reset_game()
148
149     def reset_game(self) -> None:
150         """ゲームの状態を初期化またはリスタートする"""
151         self.player: Player = Player(float(SCREEN_WIDTH
152         self.enemies: list[Enemy] = []
153         self.elapsed_frames: int = 0
154         self.spawn_timer: int = 0
155         self.game_state: str = "PLAYING"  # "PLAYING", "
156
157     def run(self) -> None:
158         """メインゲームループを実行する"""
```

```
159             running: bool = True
160             while running:
161                 # イベント処理
162                 for event in pygame.event.get():
163                     if event.type == pygame.QUIT:
164                         running = False
165                     elif event.type == pygame.KEYDOWN:
166                         if event.key == pygame.K_ESCAPE:
167                             running = False
168                         elif event.key == pygame.K_r:
169                             # ゲームオーバーまたはクリア時にRキ
170                             if self.game_state in ["GAME_OVE
171                                 self.reset_game()
172 
173                 keys = pygame.key.get_pressed()
174 
175                 if self.game_state == "PLAYING":
176                     # プレイヤーの更新
177                     self.player.update(keys)
178 
179                     # 時間経過のカウント
180                     self.elapsed_frames += 1
181                     survived_seconds = self.elapsed_frames /
182                     if survived_seconds >= TIME_LIMIT:
183                         self.game_state = "CLEAR"
184 
185                     # 一定時間ごと（約1.5秒おごと）に敵を出現さ
186                     self.spawn_timer += 1
187                     if self.spawn_timer >= 90:
188                         self.spawn_timer = 0
189                         self.spawn_enemy()
190 
191                     # 敵の更新
192                     px, py = self.player.get_pos()
193                     for enemy in self.enemies:
194                         enemy.update(px, py)
195 
196                     # 当たり判定のチェック
197                     self.check_collisions()
198 
199                     # HPが0になったらゲームオーバー
200                     if self.player.get_hp() <= 0:
201                         self.game_state = "GAME_OVER"
202 
203                 # 描画処理
204                 self.draw()
205 
206                 self.clock.tick(FPS)
207 
208             pygame.quit()
209             sys.exit()
210 
211         def spawn_enemy(self) -> None:
212             """画面の上下左右の端からランダムに敵を生成する"""
213             side = random.randint(0, 3)
214             if side == 0:  # 上端
215                 x = random.randint(0, SCREEN_WIDTH)
216                 y = -20
```

```
            elif side == 1:  # 下端
                x = random.randint(0, SCREEN_WIDTH)
                y = SCREEN_HEIGHT + 20
            elif side == 2:  # 左端
                x = -20
                y = random.randint(0, SCREEN_HEIGHT)
            else:  # 右端
                x = SCREEN_WIDTH + 20
                y = random.randint(0, SCREEN_HEIGHT)

            self.enemies.append(Enemy(float(x), float(y)))

        def check_collisions(self) -> None:
            """プレイヤーとすべての敵の接触判定を行う"""
            px, py = self.player.get_pos()
            p_radius = self.player.radius

            for enemy in self.enemies:
                ex, ey = enemy.get_pos()
                e_radius = enemy.radius
                distance = math.hypot(px - ex, py - ey)

                # 円同士の当たり判定
                if distance < p_radius + e_radius:
                    self.player.take_damage(20)  # 接触時に20

        def draw(self) -> None:
            """画面全体の描画を行う"""
            # 背景描画
            self.screen.fill(COLOR_BG)

            # キャラクター描画
            self.player.draw(self.screen)
            for enemy in self.enemies:
                enemy.draw(self.screen)

            # UI（HP、スコア、残り時間）の描画
            hp_text = self.font.render(f"HP: {self.player.ge
            self.screen.blit(hp_text, (20, 20))

            survived_seconds = self.elapsed_frames // FPS
            score_text = self.font.render(f"Score (Time): {s
            self.screen.blit(score_text, (330, 20))

            remaining_seconds = max(0, TIME_LIMIT - survived
            time_text = self.font.render(f"Limit: {remaining
            self.screen.blit(time_text, (730, 20))

            # ゲーム終了時のオーバーレイとメッセージ
            if self.game_state == "GAME_OVER":
                overlay = pygame.Surface((SCREEN_WIDTH, SCRE
                overlay.fill((0, 0, 0, 160))
                self.screen.blit(overlay, (0, 0))

                game_over_text = self.large_font.render("GAM
                restart_text = self.font.render("Press R to

                self.screen.blit(game_over_text, (SCREEN_WID
                self.screen.blit(restart_text, (SCREEN_WIDTH
```

```
276
277             elif self.game_state == "CLEAR":
278                 overlay = pygame.Surface((SCREEN_WIDTH, SCRE
279                 overlay.fill((0, 0, 0, 160))
280                 self.screen.blit(overlay, (0, 0))
281
282                 clear_text = self.large_font.render("CLEAR!"
283                 restart_text = self.font.render("Press R to
284
285                 self.screen.blit(clear_text, (SCREEN_WIDTH /
286                 self.screen.blit(restart_text, (SCREEN_WIDTH
287
288             pygame.display.flip()
289
290
291     if __name__ == "__main__":
292         game = Game()
293         game.run()
```